EcoBuck Smart Compost Monitoring System

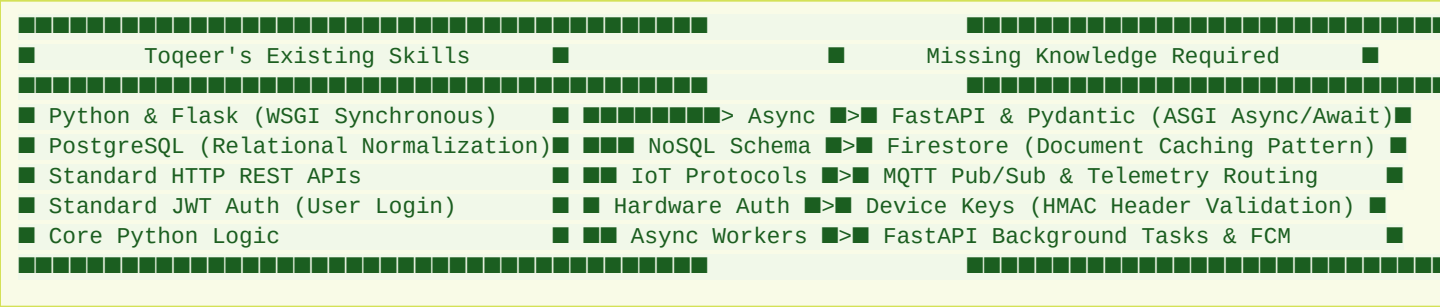
Personal Learning Roadmap & Skill-Gap Analysis for Toqeer Ahmed

Author: Toqeer Ahmed
Role: Principal IoT Systems & Backend Architect
Date: July 8, 2026
Status: PROPOSED (Learning Review Stage)

---

1. Skill Gap Analysis & Transition Strategy

Toqeer, you already possess a robust foundation in backend development, system design, relational databases (PostgreSQL), and advanced AI/ML/DL engineering. This will accelerate your implementation of the EcoBuck backend. However, shifting from a transactional relational backend to a high-concurrency, real-time IoT architecture requires a few structural transitions:



---

2. Topic-by-Topic Learning Roadmap

2.1 FastAPI

- **Why It Is Needed:** Shifting from Flask (synchronous/WSGI) to FastAPI (asynchronous/ASGI) allows the backend to handle thousands of concurrent connections from IoT devices without blocking execution threads. FastAPI also provides native Dependency Injection and automatic OpenAPI

(Swagger) documentation generation.

- **Estimated Learning Time:** 6–8 hours.

  Best Learning Resources:

  - FastAPI Official Tutorial User Guide (Highly recommended: read sections on Async, Path Params, and Dependecies).
  - *Video Guide:* "FastAPI - Modern Python Web Framework" by FreeCodeCamp.

- **Mini Project:** Build an asynchronous REST API server with three endpoints that read and write data to a local dictionary using `async`/`await` and custom dependencies.
- **Expected Outcome:** Deep understanding of the ASGI request lifecycle, dependency injection (`Depends`), and asynchronous route handling.

2.2 Pydantic

- **Why It Is Needed:** Essential for data validation and parsing. Pydantic enforces strict type safety on incoming JSON payloads at runtime and generates OpenAPI schemas automatically.
- **Estimated Learning Time:** 3–4 hours.

  Best Learning Resources:

  - Pydantic V2 Documentation - Welcome Guide.
  - *Article:* "Pydantic V2: The Future of Data Validation in Python" on RealPython.

- **Mini Project:** Write Pydantic models validating a complex nested telemetry JSON payload, including custom constraints (e.g. checking temperature limits, formatting Unix timestamps, checking enum states).
- **Expected Outcome:** Ability to write nested serializers, declare fields constraints, handle custom data validators, and use `BaseSettings` to manage environment variables.

2.3 Firebase & Firestore

- **Why It Is Needed:** Google Cloud Firestore is the selected NoSQL database for EcoBuck. You need to learn how Firestore scales, how database reads/writes are billed, and how document/collection structures differ from relational tables.
- **Estimated Learning Time:** 8–10 hours.

  Best Learning Resources:

  - Google Firestore Official Documentation.

    - *Video Series:* "Firestore Document Database - Firebase Fundamentals" on Google Developers Channel.

- **Mini Project:** Build a Python console app connecting to a free Firebase project using the `firebase-admin` SDK, performing CRUD actions on a simulated `/devices` collection and query sorting.
- **Expected Outcome:** Proficiency in structuring document hierarchies, write-duplication caching patterns, NoSQL compound filtering, and real-time listeners.

2.4 MQTT (Message Queuing Telemetry Transport)

- **Why It Is Needed:** The lightweight, pub/sub protocol widely used in IoT to minimize message overhead and battery drainage. Understanding MQTT is crucial for designing future telemetry transport upgrades.
- **Estimated Learning Time:** 4–6 hours.

  Best Learning Resources:

    - HiveMQ MQTT Essentials Guide.
    - *Python Client:* `paho-mqtt` library documentation on PyPI.

- **Mini Project:** Run a local Eclipse Mosquitto broker inside Docker, write a Python publisher script pushing telemetry data, and a subscriber script capturing the message and writing it to a log file.
- **Expected Outcome:** Understanding broker concepts, topics structuring (e.g., `ecobuck/{id}/telemetry`), Quality of Service (QoS 0/1/2) levels, and keeping client sessions alive.

2.5 Device Authentication

- **Why It Is Needed:** Standard web auth uses cookies or user session JWTs. IoT devices require lightweight, secure authentication (like API Keys, HMAC-SHA256 signatures, or client-side certificates) that does not require human login prompts.
- **Estimated Learning Time:** 4 hours.

  Best Learning Resources:

    - OWASP IoT Security Guidance - Device Identity.
    - *Article:* "Securing APIs with API Keys vs HMAC" on developer blogs.

- **Mini Project:** Write a FastAPI decorator/dependency that intercepts requests, checks for a custom API token header, verifies it using HMAC, and yields the authenticated device ID.

- **Expected Outcome:** Ability to write secure authentication endpoints for machine-to-machine integrations.

2.6 IoT Architecture & Telemetry Systems

- **Why It Is Needed:** Outlines core IoT design practices: handling device registries, heartbeat tracking, data ingestion throttling, sleep intervals, and clock drifts.
- **Estimated Learning Time:** 6 hours.

  Best Learning Resources:

  - AWS IoT Lens Whitepaper (System architecture patterns for IoT).
  - *Reference Book:* "Designing Data-Intensive Applications" by Martin Kleppmann (Chapters on stream processing and data encoding).

- **Mini Project:** Design a virtual Device Registry system tracking device state (online/offline) using dynamic heartbeat checks and timestamp validations.
- **Expected Outcome:** Mastery of device lifecycles, connection management patterns, and system scaling layouts.

2.7 Time-Series Data Modeling

- **Why It Is Needed:** Sensor telemetry generates continuous time-indexed data. You must model this data to support fast queries for historical charts without overloading database queries.
- **Estimated Learning Time:** 5 hours.

  Best Learning Resources:

  - TimescaleDB Documentation - What is Time-Series?.
  - *NoSQL Pattern Guides:* "Time-series data modeling in Firestore/NoSQL".

- **Mini Project:** Model a collection of temperature points. Write queries retrieving hourly aggregations over a 7-day range, preventing raw scans of high-frequency data.
- **Expected Outcome:** Understanding time-series modeling, downsampling aggregation pipelines, and composite index configurations.

2.8 API Security (IoT Bounds)

- **Why It Is Needed:** Secures API interfaces against DDoS attacks, brute-force requests, cross-site leaks, and unauthorized sniffing.
- **Estimated Learning Time:** 4 hours.

  Best Learning Resources:

  - FastAPI Advanced Security configurations.
  - OWASP API Security Top 10.

- **Mini Project:** Configure security headers and write rate-limiting middleware (using `slowapi` or a custom sliding-window cache) limiting telemetry uploads to max 2 per minute per key.
- **Expected Outcome:** Secure API development skills, rate limiting integration, CORS policies configurations, and security header control.

2.9 Background Tasks

- **Why It Is Needed:** Critical anomalies, push alerts, or data processing checks should run asynchronously. Running these checks on main request threads slows down device telemetry responses.
- **Estimated Learning Time:** 3 hours.

  Best Learning Resources:

  - FastAPI Background Tasks official guide.
  - *Task Queue Guide:* Celery or RQ (Redis Queue) documentation for scaling.

- **Mini Project:** Write an API route that accepts a telemetry payload, returns HTTP 202 immediately, and spawns a background thread validating thresholds and simulated emails.
- **Expected Outcome:** Ability to write responsive APIs by offloading heavy computational tasks to non-blocking background workers.

2.10 Cloud Deployment (Docker & Container Services)

- **Why It Is Needed:** Your backend must be packaged and deployed to a cloud server environment so the firmware and mobile applications can access it.
- **Estimated Learning Time:** 6 hours.

  Best Learning Resources:

  - Docker Official Getting Started Guide.
  - *Cloud Deploy:* Google Cloud Run or AWS ECS container hosting tutorials.

- **Mini Project:** Write a multi-stage `Dockerfile`, build it locally, run it inside a docker container, and publish the container to a free cloud hosting service (e.g. Google Cloud Run).
- **Expected Outcome:** Mastery of multi-stage Docker configurations, environment mapping, container security contexts, and container orchestrations.